

Computer Vision Lab 04 – Code Cheat Sheet

How to use this sheet in the coding exam

For every code block, first identify:

- INPUT: what image/data the code receives
 - OPERATION: what function is doing the actual CV work
 - OUTPUT: image, histogram, feature vector, or edge map
 - PARAMETERS: what you would change if the question changes
-

1. HOG – Histogram of Oriented Gradients

Manual pages: 6–7 (PDF pages 7–8) **Purpose:** Extract gradient/edge-orientation features and form a HOG feature vector.

```
from skimage.feature import hog
from skimage import exposure
import cv2
import matplotlib.pyplot as plt

# Read an image (grayscale)
image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)

# Compute HOG features
features, hog_image = hog(image, pixels_per_cell=(8, 8),
                           cells_per_block=(2, 2), visualize=True)

# Rescale the HOG image for better visualization
hog_image_rescaled = exposure.rescale_intensity(hog_image, in_range=(0, 10))

# Plot the HOG image and its corresponding feature vector
plt.figure(figsize=(12, 6))
plt.subplot(121)
plt.imshow(image, cmap=plt.cm.gray)
plt.title('Original Image')

plt.subplot(122)
plt.imshow(hog_image_rescaled, cmap=plt.cm.gray)
plt.title('HOG Features')
plt.show()
```

```
# Print the HOG feature vector
print("HOG Feature Vector:")
print(features)
```

Function / argument dictionary

- `cv2.imread(filename, flags)`
 - `filename` = image file/path
 - `flags` = how to read it; manual uses `cv2.IMREAD_GRAYSCALE`
- `hog(image, pixels_per_cell=..., cells_per_block=..., visualize=...)`
 - `image` = input image
 - `pixels_per_cell` = cell size; manual uses `(8, 8)`
 - `cells_per_block` = number of cells in each block; manual uses `(2, 2)`
 - `visualize=True` = also return a HOG visualization
 - returns `features, hog_image`
- `exposure.rescale_intensity(image, in_range=...)`
 - rescales intensity values for better display
- `plt.figure(figsize=(w,h))`
 - creates plotting figure; manual uses `(12, 6)`
- `plt.subplot(rows, cols, index)`
 - selects a subplot position
 - `121` = 1 row, 2 columns, first plot
 - `122` = second plot
- `plt.imshow(image, cmap=...)`
 - displays an image
- `plt.title(text)` = plot title
- `plt.show()` = display plots
- `print(value)` = print the feature vector

Exam modifications

- Change image path: `'image2.jpg'`
- Change cell size: `pixels_per_cell=(16, 16)`
- Change block size: `cells_per_block=(3, 3)`
- Remove visualization: `visualize=False`

2. LBP — Local Binary Pattern

Manual pages: 8–9 (PDF pages 8–9) **Purpose:** Extract local texture patterns and use their histogram as a feature vector.

```

from skimage import feature
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read an image (grayscale)
image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)

# Compute LBP features
radius = 1 # Radius of the circular neighborhood
n_points = 8 * radius # Number of neighboring pixels to consider

lbp_image = feature.local_binary_pattern(image, n_points, radius,
                                         method='uniform')

# Calculate the LBP histogram
hist, _ = np.histogram(lbp_image.ravel(), bins=np.arange(0,
                                                         n_points + 3), range=(0, n_points + 2))

# Normalize the histogram
hist = hist.astype("float")
hist /= (hist.sum() + 1e-6)

# Display the LBP image and histogram
plt.figure(figsize=(12, 6))
plt.subplot(121)
plt.imshow(lbp_image, cmap='gray')
plt.title('LBP Image')

plt.subplot(122)
plt.bar(range(0, n_points + 2), hist)
plt.title('LBP Histogram')
plt.xlabel('LBP Patterns')
plt.ylabel('Frequency')
plt.show()

# Print the LBP feature vector
print("LBP Feature Vector:")
print(hist)

```

Function / argument dictionary

- `feature.local_binary_pattern(image, P, R, method=...)`
 - `image` = grayscale input
 - `P` = number of sampling points/neighbors; manual uses `n_points`
 - `R` = radius; manual uses `radius`
 - `method='uniform'` = uniform LBP method used by the manual
- `lbp_image.ravel()` = flatten array into 1-D

- `np.arange(start, stop)`
 - creates evenly spaced integer values; manual uses it to make histogram bins
- `np.histogram(a, bins=..., range=(low, high))`
 - `a` = values to count
 - `bins` = number/locations of bins
 - `range` = lower/upper range included
 - returns histogram and bin edges
- `hist.astype("float")` = convert histogram datatype to float
- `hist.sum()` = sum all histogram values
- `1e-6` = tiny value added to avoid division by zero
- `plt.bar(x, height)` = bar chart

Exam modifications

- `radius = 2` → `n_points = 16`
- Change `method` only if sir specifically asks for another method
- Keep the histogram + normalization pattern

3. Histogram of Color

Manual page: 9 (PDF page 10) Important: The manual gives the **explanation and working mechanism**, but there is **no standalone Python code block for Histogram of Color** in the provided 23-page manual. Do not invent a “manual code” that is not actually there.

What the manual says: divide the color space into bins (e.g. RGB or HSV) and count pixels in each bin.

Exam recognition

Color image → color bins → pixel counts → color histogram → feature representation

4. HED — Histogram of Edge Directions

Manual pages: 10–11 (PDF pages 10–11) Purpose: Detect edges, calculate their orientations, bin the orientations, and use the histogram as the HED feature vector.

```
import cv2
import numpy as np
```

```

import matplotlib.pyplot as plt

# Read an image (grayscale)
image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)

# Apply Gaussian smoothing to reduce noise (optional)
image_smoothed = cv2.GaussianBlur(image, (5, 5), 0)

# Apply Canny edge detection
edges = cv2.Canny(image_smoothed, threshold1=30, threshold2=70)

# Calculate gradient direction (orientation)
gradient_x = cv2.Sobel(image_smoothed, cv2.CV_64F, 1, 0, ksize=3)
gradient_y = cv2.Sobel(image_smoothed, cv2.CV_64F, 0, 1, ksize=3)
gradient_magnitude = np.sqrt(gradient_x**2 + gradient_y**2)
gradient_orientation = np.arctan2(gradient_y, gradient_x) * 180 / np.pi

# Create a histogram of edge directions
hist, bin_edges = np.histogram(gradient_orientation, bins=8, range=(0, 360))

# Display the edge map and the histogram
plt.figure(figsize=(12, 6))
plt.subplot(121)
plt.imshow(edges, cmap='gray')
plt.title('Edge Map')

plt.subplot(122)
plt.bar(bin_edges[:-1], hist, width=45, align='center')
plt.title('Histogram of Edge Directions')
plt.xlabel('Edge Direction (degrees)')
plt.ylabel('Frequency')
plt.xticks(range(0, 360, 45))
plt.show()

# Print the HED feature vector (histogram)
print("Histogram of Edge Directions (HED) Feature Vector:")
print(hist)

```

Function / argument dictionary

- `cv2.GaussianBlur(src, ksize, sigmaX)`
 - `src` = input image
 - `ksize` = Gaussian kernel size; manual uses (5,5)
 - `sigmaX` = Gaussian sigma in x direction; manual uses 0
- `cv2.Canny(image, threshold1, threshold2)`
 - `threshold1` = lower threshold; manual uses 30
 - `threshold2` = upper threshold; manual uses 70
- `cv2.Sobel(src, ddepth, dx, dy, ksize)`

- `ddepth = output depth; manual uses cv2.CV_64F`
- `dx=1, dy=0` → x-gradient
- `dx=0, dy=1` → y-gradient
- `ksize=3` → 3×3 Sobel kernel
- `np.sqrt(expression)` = square root; used for gradient magnitude
- `np.arctan2(y, x)` = gradient angle/orientation in radians
- `* 180 / np.pi` = radians → degrees
- `np.histogram(values, bins=8, range=(0,360))` = 8-bin orientation histogram over 0–360 degrees
- `plt.bar(x, height, width=..., align=...)` = bar histogram plot
- `plt.xticks(values)` = choose x-axis tick locations

Exam modifications

- Change Canny thresholds.
- Change orientation bins, e.g. `bins=12`.
- Change the orientation range only if sir asks.
- Remember: in this manual HED is **Histogram of Edge Directions**.

5. HIG — Histogram of Intensity Gradients

Manual pages: 12–13 (PDF pages 12–13) **Purpose:** Compute intensity gradients, make an orientation histogram, normalize it, and use it as a feature vector.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read an image (grayscale)
image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)

# Compute the gradient using the Sobel operator
dx = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=3)
dy = cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=3)

# Calculate the magnitude and orientation of the gradient
magnitude = np.sqrt(dx**2 + dy**2)
orientation = np.arctan2(dy, dx) * (180 / np.pi) # Convert radians to degrees

# Create a histogram of gradient orientations
histogram, bins = np.histogram(orientation, bins=9, range=(0, 180))

# Normalize the histogram (optional)
```

```

histogram = histogram / histogram.sum()

# Display the histogram
plt.bar(bins[:-1], histogram, width=20)
plt.title("Histogram of Intensity Gradients (HIG)")
plt.xlabel("Gradient Orientation (degrees)")
plt.ylabel("Frequency")
plt.show()

# You can use the computed histogram as a feature vector for your application.

```

Function / argument dictionary

Same core functions as HED, but note the manual's HIG code uses:

- `np.histogram(orientation, bins=9, range=(0,180))`
 - 9 bins
 - orientation range 0–180 degrees
- `histogram / histogram.sum() = normalization`
- `plt.bar(bins[:-1], histogram, width=20) = display histogram`

Critical distinction

- **HED manual code:** `bins=8, range=(0,360)`
- **HIG manual code:** `bins=9, range=(0,180)`

6. Texture Energy and Contrast Histograms

Manual pages: 14–15 (PDF pages 14–15) **Purpose:** Extract local texture energy/contrast information and build histograms that can be used as feature vectors.

```

import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read an image (grayscale)
image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)

# Define the neighborhood size (e.g., 3x3 window)
neighborhood_size = 3

# Calculate texture energy and contrast using OpenCV filter2D
energy = cv2.filter2D(image ** 2, -1, np.ones((neighborhood_size, neighborhood_size)))

```

```

contrast = cv2.filter2D(image, -1, np.ones((neighborhood_size, neighborhood_size)))

# Compute histograms for texture energy and contrast
energy_histogram, energy_bins = np.histogram(energy, bins=256,
                                             range=(0, energy.max()))
contrast_histogram, contrast_bins = np.histogram(contrast, bins=256,
                                                range=(0, contrast.max()))

# Normalize histograms (optional)
energy_histogram = energy_histogram / energy_histogram.sum()
contrast_histogram = contrast_histogram / contrast_histogram.sum()

# Display the histograms
plt.figure(figsize=(12, 4))
plt.subplot(2, 1, 1)
plt.plot(energy_bins[:-1], energy_histogram, color='b')
plt.title("Texture Energy Histogram")
plt.xlabel("Energy")
plt.ylabel("Frequency")

plt.subplot(2, 1, 2)
plt.plot(contrast_bins[:-1], contrast_histogram, color='r')
plt.title("Texture Contrast Histogram")
plt.xlabel("Contrast")
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()

# You can use the computed histograms as feature vectors for texture analysis.

```

Function / argument dictionary

- `np.ones(shape)`
 - creates an array of ones
 - manual uses `(neighborhood_size, neighborhood_size)` → e.g. `(3,3)`
- `cv2.filter2D(src, ddepth, kernel)`
 - `src` = input image/array
 - `ddepth` = desired output depth; manual uses `-1`
 - `kernel` = filter/kernel
- `image ** 2` = square every pixel value before filtering for the manual's energy calculation**
- `energy.max()` / `contrast.max()` = maximum value, used to define histogram range
- `plt.plot(x, y, color=...)` = line plot
- `plt.tight_layout()` = automatically adjusts subplot spacing

Important manual note

The written theory defines texture contrast as the **standard deviation of pixel intensities in a local neighborhood**, while the supplied code block calculates contrast using `cv2.filter2D(image, ...)`. For an exam asking for the **manual's code**, reproduce the supplied code; for a theory question, use the manual's stated definition.

7. Box Blur / Simple Blurring with filter2D

Manual page: 16 (PDF page 16) **Purpose:** Average neighboring pixels for smoothing/noise reduction.

```
import cv2
import numpy as np

# Read an image
image = cv2.imread('image.jpg')

# Define a box blur kernel
kernel = np.ones((3, 3), dtype=np.float32) / 9

# Apply convolution
blurred_image = cv2.filter2D(image, -1, kernel)
```

Function / argument dictionary

- `np.ones((3,3), dtype=np.float32)`
 - `(3,3)` = kernel size
 - `dtype` = datatype
- `/ 9`
 - average of 9 kernel values
- `cv2.filter2D(src, ddepth, kernel)`
 - applies kernel/filter to image
 - `ddepth=-1` = output image depth same as source in this usage

Exam changes

- 5×5 blur: `np.ones((5,5), dtype=np.float32) / 25`
 - Change kernel values if asked for a custom filter.
-

8. Gaussian Blur

Manual page: 17 (PDF page 17) **Purpose:** Smooth the image using a Gaussian-shaped kernel; manual notes it is often used for noise reduction.

```
# Define a Gaussian blur kernel
kernel_size = (5, 5)
sigma = 1.0

kernel = cv2.getGaussianKernel(kernel_size[0], sigma)
gaussian_blur = cv2.filter2D(image, -1, kernel)
```

Function / argument dictionary

- `cv2.getGaussianKernel(ksize, sigma)`
 - `ksize` = kernel size in the direction used; manual passes `kernel_size[0] = 5`
 - `sigma` = Gaussian standard deviation; manual uses `1.0`
- `cv2.filter2D(image, -1, kernel)` = apply generated kernel

Exam recognition

Gaussian blur → smoothing → noise reduction

9. Sobel and Scharr Edge Detection

Manual page: 17 (PDF page 17) **Purpose:** Detect edges by emphasizing rapid changes in pixel values.

```
# Sobel edge detection
sobel_x = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=3)

# Scharr edge detection
scharr_x = cv2.Scharr(image, cv2.CV_64F, 1, 0)
scharr_y = cv2.Scharr(image, cv2.CV_64F, 0, 1)
```

Function / argument dictionary

- `cv2.Sobel(src, ddepth, dx, dy, ksize)`
 - `dx=1, dy=0` → x/horizontal derivative
 - `dx=0, dy=1` → y/vertical derivative
 - `ksize=3` → 3×3
- `cv2.Scharr(src, ddepth, dx, dy)`

- same `dx/dy` direction idea
- manual code does not pass `ksize`

10. Embossing

Manual page: 17 (PDF page 17) **Purpose:** Create a 3D-like/embossed visual effect by emphasizing neighboring-pixel differences.

```
# Define an emboss kernel
kernel = np.array([[ -2, -1,  0],
                  [ -1,  1,  1],
                  [  0,  1,  2]], dtype=np.float32)

embossed_image = cv2.filter2D(image, -1, kernel)
```

Function / argument dictionary

- `np.array(object, dtype=...)`
 - `object` = nested list containing kernel values
 - `dtype=np.float32` = floating-point kernel
- `cv2.filter2D(...)` = applies emboss kernel

Exam recognition

Custom kernel + `filter2D` = custom image effect/filter.

11. Standard Convolution

Manual pages: 17–18 (PDF pages 18–19) **Purpose:** Apply a custom 3×3 kernel to the image.

```
import cv2
import numpy as np

# Read an image
image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)

# Define a kernel (3x3 example)
kernel = np.array([[1, 0, -1],
                  [2, 0, -2],
                  [1, 0, -1]], dtype=np.float32)
```

```
# Apply standard convolution
result = cv2.filter2D(image, -1, kernel)
```

Function / argument dictionary

- `np.array([...], dtype=np.float32)` = creates the custom kernel
- `cv2.filter2D(src, ddepth, kernel)` = applies kernel

Kernel to recognize

```
1  0 -1
2  0 -2
1  0 -1
```

This is the kernel used by the manual's standard-convolution example.

12. Valid Convolution – No Padding

Manual page: 18 (PDF page 18) **Purpose:** Manual describes valid/no-padding convolution as processing only where the kernel fully overlaps the image, producing a smaller output.

```
import cv2
import numpy as np

# Read an image
image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)

# Define a kernel (3x3 example)
kernel = np.array([[1, 0, -1],
                   [2, 0, -2],
                   [1, 0, -1]], dtype=np.float32)

# Apply valid convolution (no padding)
result = cv2.filter2D(image, -1, kernel, borderType=cv2.BORDER_CONSTANT)
```

Function / argument dictionary

- `cv2.filter2D(src, ddepth, kernel, borderType=...)`
- `cv2.BORDER_CONSTANT` = constant-value border handling in the supplied code

Theory to memorize

Valid / no padding → kernel must fully overlap → smaller output

13. Same Convolution – Zero Padding

Manual pages: 18–19 (PDF pages 18–19) **Purpose:** Manual says same convolution uses padding to keep the output dimensions the same as the input.

```
import cv2
import numpy as np

# Read an image
image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)

# Define a kernel (3x3 example)
kernel = np.array([[1, 0, -1],
                   [2, 0, -2],
                   [1, 0, -1]], dtype=np.float32)

# Apply same convolution (zero padding)
result = cv2.filter2D(image, -1, kernel, borderType=cv2.BORDER_REFLECT)
```

Function / argument dictionary

- `borderType=cv2.BORDER_REFLECT` = reflect border pixels in the supplied code

Important source-specific note

The section is titled “**Same Convolution (Zero Padding)**”, but the supplied code uses `cv2.BORDER_REFLECT`. In the exam, distinguish the **concept described by the manual** from the **exact borderType used in its code**.

14. Valid Convolution with Strides

Manual page: 19 (PDF page 19) **Purpose:** Valid convolution where the kernel skips positions according to a stride; manual says the output becomes smaller.

```
import cv2
import numpy as np

# Read an image
```

```

image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)

# Define a kernel (3x3 example)
kernel = np.array([[1, 0, -1],
                   [2, 0, -2],
                   [1, 0, -1]], dtype=np.float32)

# Apply valid convolution with a stride of 2
result = cv2.filter2D(image, -1, kernel, strides=(2, 2),
                     borderType=cv2.BORDER_CONSTANT)

```

Function / argument dictionary

- `strides=(2,2)` in the manual = intended to specify movement of 2 pixels in both directions
- `borderType=cv2.BORDER_CONSTANT` = border handling shown in the supplied code

Exam theory

Stride 1 → move normally
 Stride 2 → skip to every second position
 Larger stride → generally smaller output / fewer kernel positions

15. Gradient-Based Edge Detection + Threshold

Manual page: 21 (PDF page 21) **Purpose:** Compute x/y gradients, gradient magnitude/direction, then threshold magnitude to create a binary edge image.

```

import cv2
import numpy as np

# Read an image in grayscale
image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)

# Apply Sobel operator to calculate gradients
gradient_x = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=3) # Gradient in the x-direction
gradient_y = cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=3) # Gradient in the y-direction

# Calculate the gradient magnitude and direction
gradient_magnitude = np.sqrt(gradient_x**2 + gradient_y**2)
gradient_direction = np.arctan2(gradient_y, gradient_x)

# Optionally, you can apply a threshold to identify edges

```

```
threshold = 100
edges = (gradient_magnitude > threshold).astype(np.uint8) * 255

# Display the original image and detected edges
cv2.imshow('Original Image', image)
cv2.imshow('Detected Edges', edges)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Function / argument dictionary

- `cv2.Sobel(...)` = calculate directional gradient
- `np.sqrt(...)` = gradient magnitude
- `np.arctan2(y,x)` = gradient direction
- `(gradient_magnitude > threshold)` = Boolean mask
- `.astype(np.uint8)` = convert True/False mask to 8-bit integer
- `* 255` = turns True values into 255 and False into 0
- `cv2.imshow(window_name, image)` = display image
- `cv2.waitKey(0)` = wait indefinitely for a key press
- `cv2.destroyAllWindows()` = close OpenCV windows

Extremely important pattern

```
magnitude > threshold
    ↓
True / False
    ↓
uint8
    ↓
×255
    ↓
binary edge image (0/255)
```

16. Canny Edge Detector

Manual page: 23 (PDF page 23) **Purpose:** Use OpenCV's Canny edge detector with low/high thresholds and display the resulting edge map.

```
import cv2

# Read an image in grayscale
image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)
```

```
# Apply edge detection
edges = cv2.Canny(image, threshold1=100, threshold2=200) # Adjust threshold values as needed

# Display the edge map
cv2.imshow('Canny Edges', edges)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Function / argument dictionary

- `cv2.Canny(image, threshold1, threshold2)`
 - `image` = grayscale input
 - `threshold1` = lower threshold; manual uses 100
 - `threshold2` = upper threshold; manual uses 200
 - `output` = Canny edge map
- `cv2.imshow(name, image)` = display
- `cv2.waitKey(0)` = wait for key
- `cv2.destroyAllWindows()` = close all OpenCV windows

Canny theory to memorize

1. Gaussian smoothing
2. Gradient calculation
3. Non-Maximum Suppression
4. Edge Tracking by Hysteresis

And for hysteresis:

<code>> T_high</code>	→ strong edge
<code>T_low-T_high</code>	→ weak edge (kept when connected appropriately)
<code>< T_low</code>	→ rejected

MASTER FUNCTION QUICK REFERENCE

Image input

```
cv2.imread(filename, flags)
```

- `filename` = path

- `flags = mode, e.g. cv2.IMREAD_GRAYSCALE`

Filtering / convolution

```
cv2.filter2D(src, ddepth, kernel, borderType=...)
```

- `src` = image/array
- `ddepth` = output depth; manual commonly uses `-1`
- `kernel` = filter matrix
- `borderType` = boundary handling when supplied

Gradient

```
cv2.Sobel(src, ddepth, dx, dy, ksize=3)  
cv2.Scharr(src, ddepth, dx, dy)
```

- `(1,0)` = x-gradient
- `(0,1)` = y-gradient

Blur

```
cv2.GaussianBlur(src, ksize, sigmaX)  
cv2.getGaussianKernel(ksize, sigma)
```

Canny

```
cv2.Canny(image, threshold1, threshold2)
```

- first threshold = low
- second threshold = high

Arrays / math

```
np.array(..., dtype=...)  
np.ones(shape)  
np.sqrt(expression)  
np.arctan2(y, x)  
np.histogram(values, bins=..., range=(...,...))  
np.arange(start, stop)
```

Plotting

```
plt.figure(figsize=(w,h))
plt.subplot(rows, cols, index)
plt.imshow(image, cmap=...)
plt.bar(x, height, width=...)
plt.plot(x, y)
plt.title(text)
plt.xlabel(text)
plt.ylabel(text)
plt.xticks(values)
plt.tight_layout()
plt.show()
```

OpenCV display

```
cv2.imshow(window_name, image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

LAB 4 “IF SIR CHANGES THE QUESTION” CHEAT TABLE

Question asks	Use / change
Read grayscale image	<code>cv2.imread('...', cv2.IMREAD_GRAYSCALE)</code>
HOG	<code>hog(image, pixels_per_cell=..., cells_per_block=..., visualize=...)</code>
LBP	<code>feature.local_binary_pattern(image, n_points, radius, method='uniform')</code>
Color distribution	Concept only in this manual; no standalone code block
Edge direction histogram	Canny + Sobel + <code>np.arctan2</code> + <code>np.histogram</code>
Intensity gradient histogram	Sobel + magnitude/orientation + <code>np.histogram</code>

Question asks	Use / change
Texture energy	<code>image ** 2 + cv2.filter2D</code> with ones kernel
Texture histogram	<code>np.histogram(...)</code>
Simple blur	ones kernel + <code>cv2.filter2D</code>
Gaussian blur	<code>cv2.getGaussianKernel</code> + <code>cv2.filter2D</code>
X edge/gradient	Sobel <code>(1,0)</code>
Y edge/gradient	Sobel <code>(0,1)</code>
Scharr X	<code>cv2.Scharr(...,1,0)</code>
Scharr Y	<code>cv2.Scharr(...,0,1)</code>
Custom filter/effect	<code>np.array(kernel) + cv2.filter2D</code>
Valid/no padding	use no-padding concept; manual code shows <code>BORDER_CONSTANT</code>
Same	manual section uses <code>BORDER_REFLECT</code>
Stride	manual code shows <code>strides=(2,2)</code>
Gradient magnitude	<code>np.sqrt(dx**2 + dy**2)</code>
Gradient direction	<code>np.arctan2(dy, dx)</code>
Binary edge threshold	<code>(magnitude > threshold).astype(np.uint8) * 255</code>
Canny	<code>cv2.Canny(image, low, high)</code>

THE 5 CODE PATTERNS I WOULD MEMORIZE FIRST

Pattern 1 — Sobel gradients

```
dx = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=3)
dy = cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=3)
```

Pattern 2 — Gradient magnitude + direction

```
magnitude = np.sqrt(dx**2 + dy**2)
direction = np.arctan2(dy, dx)
```

Pattern 3 — Threshold into edge image

```
edges = (magnitude > threshold).astype(np.uint8) * 255
```

Pattern 4 — Canny

```
edges = cv2.Canny(image, threshold1=100, threshold2=200)
```

Pattern 5 — Custom convolution/filter

```
kernel = np.array([...], dtype=np.float32)
result = cv2.filter2D(image, -1, kernel)
```

LAB 4 TASK CONNECTION

For the material-classification task, the manual specifically asks for at least two texture feature extraction techniques, how they work, suitable texture examples, and advantages/limitations. The most directly reusable coded techniques from this manual are **LBP** and **Texture Energy/Contrast Histograms**.